工程治理与项目组织：让项目真正做成

能写代码只是起点，能让项目在约束下可靠交付，才是工程能力。

代码能跑，项目为什么仍会失败？

本讲把失败原因拆开

- 不是只看技术实现，而是看约束下能否交付
- 从“能写代码”进入“能组织项目”
- 最终产出：可行性摘要、计划初稿、风险清单

判断
项目是否值得启动

拆分
目标是否能变成任务

跟踪
哪些任务会拖住全局

兜底
风险发生前如何准备退路

课堂主线：技术可行 ≠ 项目可行；项目治理负责把系统放进真实约束中。

本讲位置：从复杂性和节奏走向项目治理

项目治理是一条能力链：先判断、再拆分、再跟踪，最后为偏离准备退路。

复杂性来源
需求、团队、技术、交付约束

→

开发节奏
瀑布、原型、迭代、敏捷如何取舍

→

工程治理
可行性、计划、协作、风险

→

需求分析
把真实问题转成可验证规格

先判断	项目是否值得启动，启动条件是否清楚
再拆分	用 WBS 把目标变成可执行任务
再跟踪	用关键路径、Ticket、PR、CI 暴露偏差
最后兜底	用风险评分和 B 计划降低失败损失

失败案例：一次看似合理的跨平台迁移

技术收益越诱人，越要提前检查技术之外的交付边界。

原有痛点

iOS 与 Android 双端维护，缺陷不一致，迭代成本高。

→

迁移动机

希望用 React Native 统一技术栈，提高复用率和协作效率。

→

实施过程

重写核心页面、封装组件、调整构建和测试流程。

→

风险暴露

上线前发现合规/授权边界未确认，方案不能按原计划交付。

课堂判断 关键不是“React Native 好不好”，而是技术收益很诱人时，更要提前检查交付边界。

风险在上线前才暴露，代价已经形成

可行性检查越晚，纠错成本越高。

启动

重写核心页面、封装组件
人力投入

→

推进

调整构建、测试适配、团队协同
协调成本增加

→

上线前

合规或授权边界才被确认
项目被叫停

→

结果

技术方案能跑，但不能交付
沉没成本形成

风险不是在最后才出现，而是在早期没有被看见。

案例诊断：失败发生在哪一层

这个失败不在“能不能写代码”，而在“做了之后能不能承受后果”。

层次	判断	理由	结论
值不值得做	通过	双端维护成本高，迁移有真实收益	值得继续评估
能不能做	通过	技术上有实现路径，团队能完成核心功能	不是主要问题
后果能否承受	未通过	合规、授权、上线边界未提前确认	项目无法交付

诊断结论：该案例不是“写不出来”，而是第三层可行性没有提前通过。

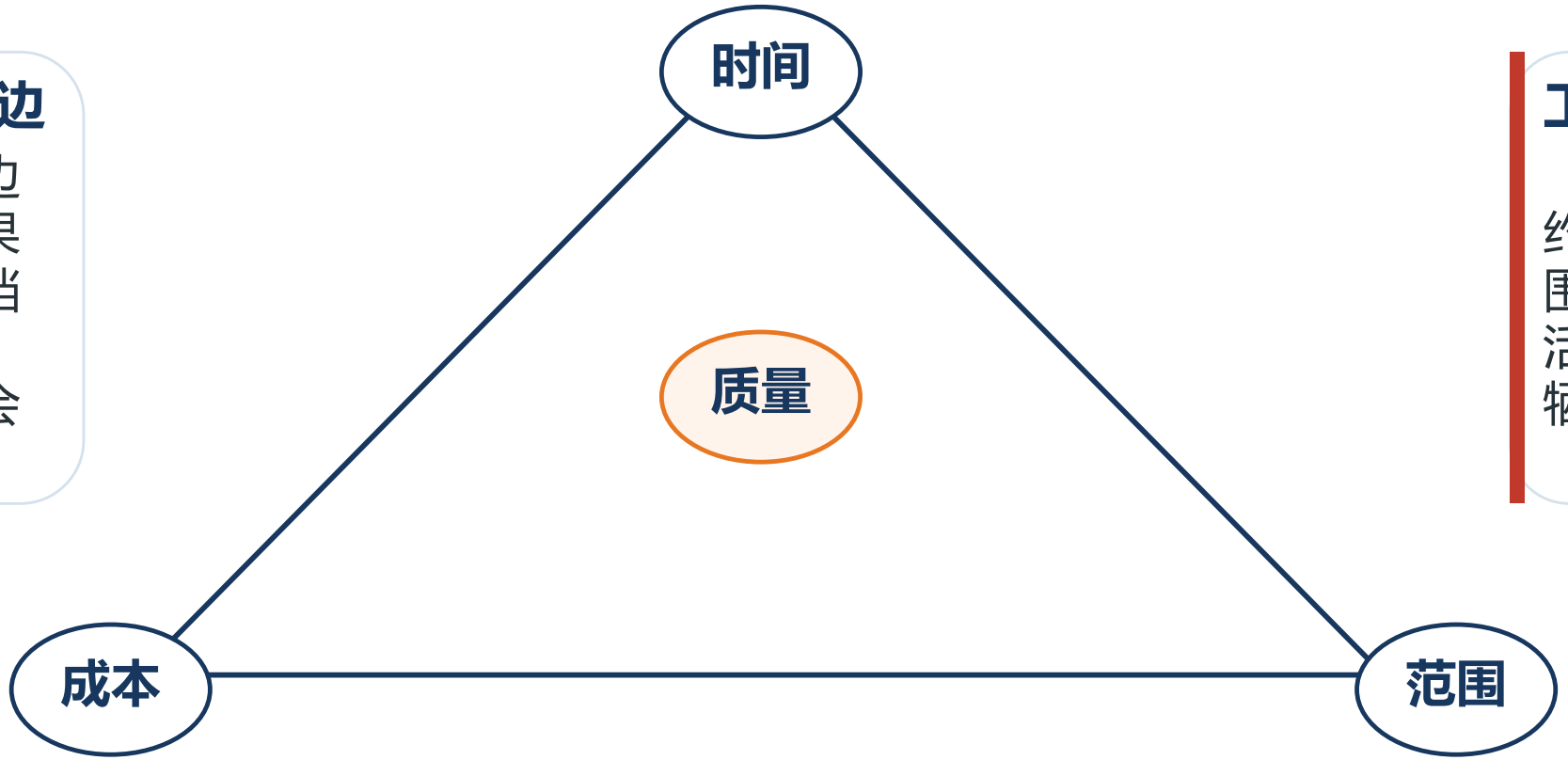
课堂动作：投票判断：这个失败主要属于哪一层可行性?

质量不是第四条边，而是三条边挤压出的结果

质量不能脱离时间、成本和范围单独保证。

不是第四条边

质量是三条边挤压后的结果：测试、文档、异常处理、可维护性都会受到影响。



工程取舍

约束不变而范围膨胀，质量活动最容易被牺牲。

3 人、10 周、43 个功能：范围压力从哪里来

范围膨胀不是多几行代码，而是同时增加设计、测试、联调、文档和交付压力。

项目参数

3 人研究生小组
10 周工期
43 个初始功能点

功能压力

登录、预约、审批、设备、通知、报表、权限、日志

隐性工作

设计、测试、联调、部署、文档、异常处理

生活类比

学院活动中节目越来越多，最先被挤掉的是彩排和应急预案。

先识别最小可演示闭环

范围管理不是把功能都删掉，而是先保证一条可验证的业务闭环。

登录 → 查看设备 → 提交预约 → 冲突检测 → 审批

必须先保证闭环
能从用户动作走到审批结果，才算可演示。

可以延后
复杂通知、统计报表、批量导出可放到后续增量。

课堂动作：圈出最小可演示版本必须包含的 4-5 个功能。

最先消失的是看不见的质量活动

测试、异常处理、文档和设计验证常常最先被牺牲，因为它们不如界面功能显眼。

可见功能

- 页面能打开
- 按钮能点击
- 数据能保存
- 演示能跑

常被牺牲的质量活动

- 边界条件测试
- 异常处理
- 日志与追溯
- 部署文档与交接

后果：重复预约、权限绕过、错误无法追溯，往往不是因为页面没做，而是质量活动被压掉。

小结：金三角回答“怎么取舍”

工程管理不是让所有目标都最大化，而是在约束下做出可解释的取舍。

范围能不能减

先做预约闭环，报表和复杂通知后移

时间能不能调

里程碑必须保护，延期要说明代价

资源能不能增

增加人手会带来沟通成本，不是万能解

质量底线是什么

权限、冲突检测、日志追溯不能牺牲

四问都能回答清楚，项目约束才算被真正纳入计划。

可行性分析：项目启动前的三层判断

项目启动前不只问“能不能写出来”，还要问“值不值得做”和“后果能不能承受”。

第一层

值不值得做

第二层

能不能做

第三层

做了能否承受后果

三层都要过，项目启动才有工程意义。

第一层：值不值得做

需求真实不等于功能越多越好，第一层要判断问题是否值得被系统化解决。

判断问题

痛点是否真实、频繁、值得处理

预约系统例子

冲突登记和沟通混乱真实存在

错误判断

只为一个实验室、少量学生做完整系统

后果

投入产出不匹配，系统维护成本高于收益

第二层：能不能做

技术可实现只是第二层判断，还必须看团队能力、资源获得和时间窗口。

技术栈

Web 系统能实现核心流程

团队能力

3 人可完成最小闭环，但不能无限扩范围

资源条件

测试账号、服务器、设备数据需要提前申请

后果

资源不到位会让计划表失去可信度

第三层：做了之后能不能承受

第三层风险往往不写在需求里，却决定系统能不能上线和长期运行。

谁维护

学生毕业后是否有人接手

数据怎么管

预约记录和借用日志涉及隐私与追溯

权限谁定

贵重设备谁能预约、谁能审批

后果

上线后无人维护，比页面不美观更致命

课堂判断：项目能不能启动

可行性分析不是否定项目，而是把启动条件和风险边界讲清楚。

任务

- 2 分钟独立判断
- 三层各写一句理由
- 给出通过/有条件通过/暂缓

现场填写

对象：实验室预约系统
结论：有条件通过
理由：先单实验室试点，不直接承诺全院上线

点评标准

- 是否按层次展开
- 是否给出具体理由
- 是否能落到可执行动作

课堂动作：完成三层判断，并写一句启动建议。

从可行性进入计划：坏计划无法执行

“完成后台开发”不是计划，因为它不能分配、不能估算、不能验收。

不可分配

不知道谁负责、边界在哪里

不可估算

不知道工作量和依赖关系

不可验收

不知道怎样证明完成

不可跟踪

只能听到“差不多完成了”

坏计划的共同点：听起来像任务，实际上无法管理。

项目计划六要素

计划不是排日历，而是建立可修正、可跟踪的工程基准线。

目标与范围
做什么，不做什么

里程碑
阶段性可检查结果

任务与责任人
谁负责交付

依赖关系
哪些任务必须先完成

时间估算
每项任务大约耗时

风险与缓冲
偏离时如何处理

好计划提供的是“可检查的基准线”，不是对未来的精确预言。

WBS：把模糊目标拆成可执行任务

WBS 的合理粒度是能分配责任人、能估算时间、能判断完成与否。

可分配

能明确责任人

可估算

能估计工作量

WBS 合理粒度

可跟踪

能看到状态变化

可验收

能证明完成

判断口诀：拆到“能分配、能估算、能跟踪、能验收”为止。

预约系统 WBS 总图

先按业务模块拆出工作结构，再用子任务具体到执行任务。

实验室预约系统
课程项目范围

账号与权限
- 登录
- 角色
- 权限校验

资源管理
- 实验室
- 设备
- 可预约时段

预约审批
- 申请
- 冲突检测
- 审批记录

通知与日志
- 站内消息
- 操作日志
- 审计追踪

审批模块：从功能拆到任务

合格的任务必须能够分配、估算、开发和验收。

功能目标

教师/实验员能够审批预约申请

→

任务包

- 审批列表接口
- 通过/驳回操作
- 审批状态流转
- 审批日志记录

→

验收标准

- 能查看待审批申请
- 审批后状态正确变化
- 异常权限不能审批
- 日志能追溯操作人

课堂练习：把“预约审批功能”拆开

能拆分成能验收的任务，才是计划中的有效任务。

任务

- 拆分对象：预约审批
- 至少写 4 个任务
- 每个任务写验收标准

现场填写

示例：
审批列表接口
审批操作接口
状态流转规则
审批日志与测试

点评标准

- 是否按层次展开
- 是否给出具体理由
- 是否能落到可执行动作

课堂动作：现场写 4 个任务，并说明每个任务如何验收。

关键路径：拖动整体工期的依赖链

关键路径不是最重要任务，而是延误会直接拖慢整体工期的最长依赖链。

统一认证 → 权限模型 → 预约提交 → 审批流程 → 联调测试

判断规则

关键路径不是“最重要的任务”，而是“任何延误都会拖动整体工期的依赖链”。

任务依赖表：先把依赖写出来

没有任务依赖表，就很难看清哪些任务可以并行，哪些任务会卡住全局。

任务	依赖	工期
A 统一认证接入	无	5 天
B 设备列表页	无	3 天
C 角色权限模型	A	4 天
D 预约提交接口	C	4 天
E 审批流程	D	5 天
F 通知模板	无	2 天
G 联调测试	E、F	3 天

课堂动作：根据表格圈出关键路径。

关键路径示例：A -> C -> D -> E -> G

关键路径上的任务需要提前验证和预留缓冲，因为它们会牵动整体交付。

关键路径工期

5 + 4 + 4 + 5 + 3 = 21 天

为什么 B、F 不在主路径上

它们可以并行完成，不直接拖动最终交付时间。

“80% 完成”为什么不可信

工程进度最可靠的状态通常只有未完成和满足验收标准。

自报 80% 完成

- 页面画好了
- 接口写了一半
- 数据有字段
- 能本地跑

实际缺口

- 接口未接
- 权限未测
- 异常未处理
- 不能联调和部署

计划是一条可修正的基准线

没有计划，项目偏离时也无法被发现。

GPS 类比

- 导航路线
- 推荐改道
- 偏航提醒
- 重新规划

项目过程

- 项目基准计划
- 风险和变更处理
- 进度跟踪
- 资源或路径调整

计划进入多人执行，协作复杂度开始上升

任务一旦分给多人，项目就不再只是技术问题，而是状态同步和协作控制问题。

3 人

沟通成本

3 条沟通通道

6 人

沟通成本

15 条沟通通道

10 人

沟通成本

45 条沟通通道

公式：n(n-1)/2。人越多，越需要任务记录、接口约定和流程规范。

流程规范解决具体协作故障

好的流程牺牲少量个人随意性，换取团队整体可控。

协作故障

- 任务边界不清
- 直接改主分支
- 合并后才发现构建失败
- 大量任务状态不透明

流程工具

- Ticket：任务可追踪
- PR：变更可评审
- CI：问题前移
- 看板：状态可见

类比：红绿灯不是为了让每个人变慢，而是让多人共享道路时不互相撞车。

Ticket：让任务变成可跟踪记录

一个好 Ticket 要让团队知道做什么、谁来做、何时完成、怎样验收。

坏 Ticket

- 完成后台开发
- 范围不清
- 无法验收
- 无法估算

好 Ticket

- 背景：预约提交失败
- 输入：设备 ID 与时间段
- 输出：申请编号与状态
- 验收：至少 3 条接口测试

PR 与 CI：把问题前移

PR 和 CI 的价值是让错误在进入主分支之前被发现。

开发分支 → PR → Review → CI → Merge

PR 的作用

代码进入主分支前先被人看见、讨论和修正。

CI 的作用

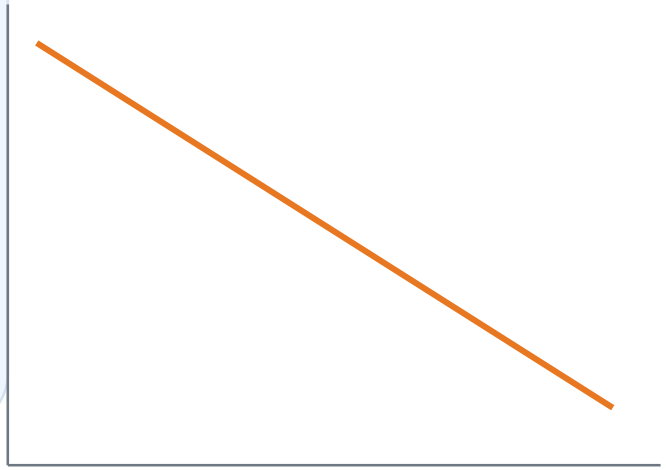
自动构建和测试，把错误挡在合并之前。

看板与燃尽图：让进度偏差可见

进度管理的关键不是问“做完了吗”，而是让阻塞、剩余工作和偏差持续可见。

WIP 限制：进行中的任务不能无限增加

燃尽趋势



剩余工作量下降变慢，说明估算、范围或阻塞出现问题。

课堂动作：判断燃尽图下降变慢，可能说明哪些问题？

风险：尚未发生但值得准备的负面事件

风险管理不是担心，而是把不确定性变成可识别、可排序、可应对的对象。

风险

- 尚未发生
- 有发生概率
- 可能造成负面影响
- 需要提前识别和排序

问题

- 已经发生
- 正在影响进度或质量
- 需要立即处置
- 处置后还要复盘

类比：体检发现高风险指标不是已经生病，而是提醒提前干预。

风险优先级 = 发生概率 × 可能损失

不是所有风险都同等处理，优先处理高概率、高损失风险。

风险优先级 = 发生概率 × 可能损失

发生概率 \ 可能损失		
3	6	9
2	4	6
1	2	3

高风险示例

统一认证：概率 3 × 损失 3 = 9

处理原则

优先处理高概率、高损失风险。

六类常见风险：从预约系统识别

风险识别要覆盖进度、人员、技术、需求、外部依赖和发布交付。

进度

第8周才开始联调

人员

成员因论文或实习退出

技术

冲突检测规则复杂

需求

指导教师持续加功能

外部依赖

认证、服务器审批不确定

发布

内网部署窗口不确定

识别风险时要覆盖进度、人员、技术、需求、外部依赖和发布交付。

风险评分表：先排队，再处理

风险表的目的不是把问题写满，而是确定处理顺序。

风险	概率	损失	得分	策略
统一认证接入失败	3	3	9	缓解 + B 计划
需求频繁增加	3	2	6	范围控制
成员临时退出	2	3	6	知识备份
通知功能推迟	2	1	2	接受或降级

课堂动作：判断哪个风险优先级最高，并说明理由。

B 计划：A 路径失败时的替代路径

B 计划不是没有信心，而是让项目在偏离 A 计划时仍有交付可能。

风险	A 计划	B 计划
统一认证失败	接入学校统一认证	本地模拟登录，保留适配层
通知服务不可用	短信或企业微信	降级为站内消息
审批规则变化	完整多级审批	先做单级审批，接口预留

统一认证风险：把关键路径、评分和 B 计划串起来

高风险外部依赖必须提前验证，因为它既在关键路径上，又可能需要 B 计划兜底。

外部依赖

统一认证
接入慢

→

关键路径

认证 -> 权限 -> 预约 -> 审批

→

风险评分

概率 3 × 损失 3 = 9

→

直接后果

权限、预约、审批全部后移

→

B 计划

本地模拟登录
保留适配层

处理策略：缓解 + B 计划。先验证外部依赖，不等到联调阶段再发现。

项目死亡信号：早期就能看见

项目失控通常不是突然发生，而是早期信号被连续忽视。

范围失控

第一版功能过大

外部依赖未验证

认证、服务器、审批接口迟迟未确认

协作透明度不足

成员进度靠口头同步

金三角失衡

需求增加但工期不变

文档未成形

核心接口和部署说明没有沉淀

死亡信号越早处理，项目越有机会回到可交付状态。

综合练习：跨校区科研设备共享平台

新项目不能只判断技术实现，还要判断组织边界和运维后果。

服务对象
不同学院、不同校区研究生

资源类型
电镜、高精度测量仪、计算工作站

审批边界
跨学院审批权、导师确认、实验员审核

责任边界
设备损坏、使用记录、数据留痕

复杂性
资源稀缺、规则多、外部依赖强

这个项目比单实验室预约系统更容易暴露第三层可行性风险。

三层可行性诊断表

综合案例题要按层次写判断，不能只写“技术可实现”。

层次	要判断的问题	判断	理由
值不值得做	是否有真实需求	学生填写	学生填写
能不能做	技术与资源是否支撑	学生填写	学生填写
后果能否承受	合规、运维、组织约束	学生填写	学生填写

课堂动作：3 分钟填写，重点写第三层至少 2 条风险。

讲评：第三层风险决定推广边界

跨组织系统最危险的地方通常不是页面和接口，而是审批权、责任边界和长期运维。

层次	参考判断
第一层	需求真实，稀缺设备共享有价值
第二层	技术可做，但规则复杂
第三层	高风险：审批权、设备损坏责任、跨学院开放意愿、数据留痕、运维交接

项目约束会反过来塑造架构选择

架构不是脱离项目管理的技术图，它必须回应团队规模、部署条件、变化频率和维护责任。

项目约束	架构含义	错误选择的代价
3 人团队	单体优先，避免过早微服务	治理成本超过开发收益
部署窗口受限	部署流程要简单稳定	发布失败后难以及时修复
需求频繁变化	核心业务接口要隔离	改动扩散到全系统
学院 IT 接手维护	技术栈可维护、文档完整	系统交接后无人敢改

架构不是“先进技术展示”，而是项目约束下的可交付选择。

M1 里程碑：把方法用到自己的项目

课后任务不是写长文档，而是形成最小可用的项目治理摘要。

交付项	最小要求	评分点
可行性分析摘要	三层可行性，每层至少 2 条判断	层次清楚、理由具体
项目计划初稿	范围、里程碑、WBS	任务可分配、可验收
关键路径判断	至少圈出 1 条依赖链	能说明为什么关键
风险清单	至少 5 个风险，含概率、损失、得分	排序合理
B 计划	至少 2 个高优先级风险的替代方案	可执行

考试与复习锚点

本讲的考试重点是能用框架分析项目，而不是背工具名词。

题型	学生动作	候选内容
名词解释	给定义 + 说明用途	金三角、WBS、关键路径、B 计划
简答题	说框架 + 结合例子	三层可行性、流程规范、风险管理
案例分析题	按层次/任务/风险写判断	可行性、计划、关键路径、风险清单

课堂动作：用一句话复述第三层可行性为什么重要。

三条必须带走的工程判断

能让项目在约束下可靠交付，才是软件工程能力。

工程判断	对应方法	回到课程项目的动作
技术可行不等于项目可行	三层可行性分析	先写启动条件和不可承受后果
计划不是排日历	WBS + 关键路径	把目标拆成能分配、能验收的任务
风险管理不是悲观	风险评分 + B 计划	为高优先级风险准备替代路径

结束问题：回到自己的项目，最需要马上检查的是范围、关键路径，还是第三层风险？